① ML Refresher

3 Ingredients

1) Hypothesis class
- model maps inputs to outputs
- Prediction function $h(x)$ + params
- Ex: Linear classifier $h_\theta(x) = \theta^T x$
  - ↳ input features, params, output vector of scores (logits)
- Ex: MNIST image → predicted digit score

2) Loss Function
- how bad prediction is
- $L(h(x), y)$ ← true label; ↑ prediction
- Ex: cross-entropy loss / softmax/classify

$$\ell_{CE}(h(x), y) = -\log(p(\text{label}=y)) = -h_y(x) + \log \sum_{j=1}^{K} e^{h_j(x)}$$

(Punish ↑ / Normalize ↑)

3) Optimization Method
- algo to find param to minimize loss
- ex: gradient descent, (SGD-minibatch)
  - GD: $\theta = \theta - \alpha \nabla_\theta f(\theta)$ (θ: param, α: lr, ∇f: gradient of loss)
    - ↳ gradient points to direction of steepest increase & move other way to min. loss
    - ↳ small α = slower learning
  - loss func differentiable so gradients won't be 0 or undefined
    - ↳ ℓ_{CE} is smooth, continuous

- Jacobian
  - matrix of partials
  - $f: \mathbb{R}^n \to \mathbb{R}^k$ so $\nabla f \in \mathbb{R}^{k \times n}$
  - numerator convention: row = output, col = in

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_k}{\partial x_1} & \cdots & \frac{\partial f_k}{\partial x_n} \end{bmatrix}$$

(* output / input)

  - ∂x: how output (loss) change wrt input (θ)
  - matrix input/output → 2D $\nabla f(x) \in 1 \times n * K \sim n \times k$

② Manual NN

- Universal func approx thm
  - a NN w/ 1 layer + enough neurons can approx any cont. func. on bounded domain well
  - simple NN can represent complex relations
  - guarantees param exist
  - layers = efficient
- Chain rule
  - $x \to f(x) \to \ell(f(x))$ • how loss changes w/ input • gradients along comp graph

$$\frac{\partial \ell}{\partial x} = \frac{\partial \ell}{\partial f} \cdot \frac{\partial f}{\partial x}$$

③ Softmax Regression

- softmax func: convert model output to probs
  - raw score (logit) → output pos, sum to 1
  - $P_i = \frac{e^{h_i(x)}}{\sum_{j=1}^{K} e^{h_j(x)}}$
- Cross-Entropy Loss: (P_i)

$$\ell_{CE}(h(x), y) = -\log(P(\text{label}=y)) = -h_y(x) + \log\left(\sum_j e^{h_j(x)}\right)$$

  - model assign high prob to correct class → small loss
  - model assign low prob to correct class → Big loss
- for multiclass classification
  - K possible labels, one correct class
  - Ex: MNIST digit classification – input dim N = 784; – K classes = 10
- logits = raw output before softmax
  - $h(x) = x\theta_n$ (input, param)
  - shape m×K (mxn nxk ⇒ m×K)
    - m = # of ex. in batch (batch size)
    - k = # of classes

⑤ Optimization

- momentum motivation
  - Vanilla GD: $\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$
    - ↳ noisy gradients from minibatch
    - ↳ oscillation in narrow valley
    - ↳ slow progress w/ gradient direction flip
  - momentum = keeps running avg of past gradients
    - ↳ move in consistent direction, smooth noise, accelerate slope
    - ↳ stabilize update

$$u_{t+1} = \beta u_t + (1-\beta)\nabla f(\theta_t) \quad \theta_{t+1} = \theta_t - \alpha u_{t+1}$$

(if $\beta = 0$, it reduces to vanilla GD)

- sparse gradient problem
  - ↳ few params get gradients during update / learn slowly
  - ↳ Ex: $\nabla f(x) = [0.8, 0, 0, 7]$ s.t 0 don't update
- ⇒ Adam: use per-parameter adaptive learning rates
  - 1st moment (mean) $u_{t+1} = \beta_1 u_t + (1-\beta_1)\nabla f(\theta_t)$
  - 2nd moment (magnitude) $v_{t+1} = \beta_2 v_t + (1-\beta_2)(\nabla f(\theta_t))^2$
  - update: $\theta_{t+1} = \theta_t - \alpha \frac{u_{t+1}}{\sqrt{v_{t+1}} + \epsilon}$ (ϵ: prevent = by 0)
  - params w/ large gradient → smaller step; small gradient → bigger step
- Exponential Moving Average
  - Weighted avg where recent values matter more
  - older terms decay exponentially
  - Momentum EMA: track avg grad. direction + history of grads
  - Adam EMA: U_t is EMA of grad (direction), V_t is EMA of squared grad (magnitude)
- Dead ReLU
  - $\text{ReLU}(x) = \max(0, x)$
  - $\text{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \le 0 \end{cases}$
  - very negative b ↓
  - ISSUE: neuron is negative for all outputs $xW + b < 0$
    - ↳ $\text{ReLU}(xW+b) = 0$
    - ↳ $\text{ReLU}'(xW+b) = 0 \Rightarrow \frac{\partial L}{\partial W} = 0 \Rightarrow$ • weights don't update, stop learning, grads stop flowing
    - * derivative of – inputs is 0
- 2nd order method – optimization using 2nd derivatives
  - Ex: Hessian matrix $H = \nabla^2 f(\theta)$ ← vs $\nabla f(\theta)$ GD
    - ↳ curvature, precise updates
    - ↳ billion parameters, expensive
    - ↳ unstable on non-convex
- optimization problems ≠ convex → few guarantees
  - NN loss funcs not convex (local minima, saddle pt, complexity)
- guarantees
  - GD converges to local minima/saddle pt (NOT global minima)
  - local just as good → success is empirical

⑥ Activation Normalization

- consistent activation scale across layers
  - → stable grads (not explode/vanish grad)
  - → easier optimization (smoother loss)
  - → less sensitive to init & lr
- $X \in \mathbb{R}^{\text{batch\_size} \times \text{dim\_feat}}$
- Layer Norm (dim_feat)
- BatchNorm (dim_feat)

- normalize across features for each example
- row-wise
- then, for each feature, scale by γ and add β
- normalize across batch for each feature
- column-wise
- avoids dependence from other ex's in batch

* BN
- Training
  - use curr_minibatch stats for mean, var
  - update running avgs
- Test
  - freeze running avgs
  - use running avgs from training

- Both have 2 learnable vector γ, β
  - ⇒ 2 * feat dim d = # model params
- LN: small batch size, sequence models (transformer), reinforcement training, not depend on batch size
- BN: large batch size, CNN/vision, ResNet, reliable stats across batch; BUT if batchsize = 1 → useless

⑦ Regularization

- Dropout
  - w/ correction: scale units not zeroed out by $\frac{1}{1-p}$
  - Training: dropout sets activations to 0 w/ prob p
    - 1-p fraction of units remain
    - E(x) expect output shrinks: $E[x_{drop}] = (1-p)E(x)$
    - Fix: scale surviving activations $E(x) = (1-p)\frac{E(x)}{1-p}$, so train=test distribution
  - prevent overfitting
    - randomly remove neurons during training
    - → robust to missing features + generalize more
- L2 Regularization
  - adds penalty to keep weights small
  - Loss = trainingLoss + $\lambda \|\theta\|^2$
    - ↳ large weights → fits noise
    - ↳ small weights → simpler model, generalize better (weight decay pushes weight to 0 if it gets big)
  - λ = strength of penalty
    - Big λ: strong reg, weights push to 0, underfit, simple
    - small λ: weak reg, big weights, overfit, minimize training loss only

③ Parameter Initialization

- Kaiming normal initialization $W \sim N(0, \frac{2}{fan_{in}})$
  - want activations + gradients to stay same scale across layers (forward / backward)
    - ↳ activation shrink → signal gone; explode → unstable
  - choice of variance σ^2 affects activation & grad. norm
    - ↳ shrink/grow w/ depth
  - For linear layer $Y = xW$, want $\text{Var}(y) \approx \text{Var}(x)$
    - ↳ if weights $W_{ij} \sim N(0, \sigma^2)$, then correct variance for relu network is $\sigma^2 = \frac{2}{n}$, n = # of inputs
    - ↳ ReLU kills ½ of activations → double var to compensate ($0 \sigma^2 = \frac{1}{n}$ → vanish grad; $\sigma^2 = \frac{3}{n}$ → explode grad)
- bias vector of linear layer
  - most b=0 b/c doesn't cause symmetry issue like weights
  - large –b = bad → dead ReLU
- weights of a linear layer
  - if W=0, neurons will produce identical output ($\begin{matrix} xW \\ x''0 \end{matrix}$)
  - same gradient same update, learn same feature
  - symmetry breaking failure
  - FIX: random init
- vanishing activation @ forward pass → small neuron output
- vanishing gradient @ backward pass → small gradient
  - ↳ early layers stop learning
  - ↳ repeated mult by small numbers
  - ↳ layer 50 grad = 0 ... layer 1 grad = 0
- exploding gradient
  - ↳ layer 50 grad = (0 ... layer 48 grad = 1000
- wrong weight variance: activation grow, backprop amplify

* BN stabilize activation
* residual connection for grad. flow

* $\hat{y} = \text{relu}(Wx+b)$

$$\nabla_W L = (\hat{y} - y)\text{ReLU}'(Wx+b) \cdot x$$

④ NN Library Design + Implementation

- static computational graph (Tensorflow 1.0)
  - define graph, build, feed input → run graph
  - pros: fast, optimized
  - cons: hard to debug / use python control flow, fixed architecture
- dynamic computation graph (pytorch, needle)
  - constructed as program runs
  - pros: easy debug, work w/ python, architecture can change
  - cons: slower, can't global optimize
- Module = basic building block of NN libraries
  - layer / model component (e.g linear layer, relu, convolution, NN)
  - forward(): y = model.forward(x), how to compute outputs
  - parameters(): return trainable weights, [W, b]
  - __init__(): initialize weights, define submodules (self.weight, self.bias)
  - Ex: relu module – input: tensor; output: max(0,x); param: None
    linear module – input: feature; output: xW; param (weight matrix)
- Sequential = convenient module chains layers together
  - Ex: Linear → ReLU → Linear → ReLU
    model = Sequential(Linear(784,128), ReLU, Linear(128,64), ReLU())
- Data Augmentation
  - ↳ transform training data randomly to make new examples
  - ↳ increase dataset, prevent overfit, generalization
  - ↳ Ex: RandomCrop / FlipHorizontal / Rotate / Resize / ColorAdjust
- Model Serving = how trained models used in production
  - online (real-time pred)
    - realtime per request (ie chatbox, fraud payment detection)
    - user req → model → response
    - Goal: low latency, fast response
    - Pros: real-time prediction, interactive, faster
    - cons: expensive, latency constraint, before execution built
  - offline (batch pred)
    - (prediction) computed in batches periodically (ie daily score)
    - dataset → batch job → predicted score
    - Goal: high throughput
    - pros: efficient, cheap, easy scale
    - cons: not real time (build @ execution)

⑩ Computer Vision

- residual (skip) connection: add input of a block to the output
  - $z_{out} = F(z_{in} + z_{in})$ (shorter flow)
  - ↳ helps gradient flow / prevent vanishing grad.
  - ↳ Easier learning – learn small corrections
- YOLO: predicts bounding boxes + classes in one pass
  - ↳ instead of sliding window takes many forward pass
  - ↳ (1) input image → convnet feature map
    (2) divide image into grid
    (3) ea. grid predict bounding box, score, class probs
  - ↳ predicts (x, y, w, h, confidence, class-probs)
    box, w×h, P(object Exist), category
- Layer Freezing: keep pretrained layers fixed during training
  - ↳ training new dataset can destroy useful learned features
  - ↳ ex: resnet on pretained image net
    - keep early layers of learned edges / textures / shapes
  - ↳ faster train, less overfit, less data needed
- transfer learning = reuse pretrained models for new task
  - early conv layers learn general visual features (edges, textures, shapes) that are useful for other vision tasks
  - YOLO uses transfer learning
    (1) classification NN on pretrained imageNet
    (2) remove classification head
    (3) add detection heads (bounding box, class prediction)
    (4) fine tune detection dataset
  - Poor if domain shift: training data ≠ new task distribution
    - Ex: animals → satellite image won't transfer well
    - also: small new dataset, diff object size, diff modalities

④ Automatic Differentiation (Backprop)

- backwards() func, the first arg out_grad = upstream gradient
  - out_grad = $\frac{\partial L}{\partial (\text{output})}$ of curr node (like Loss)
- Adjoint = gradient of final output wrt that node
  - $\bar{v}_i = \frac{\partial y}{\partial v_i}$ (how much loss changes wrt v)
  - can be calculated w/ local info b/c CHAIN RULE
    $\frac{\partial L}{\partial \text{input}} = \frac{\partial L}{\partial \text{output}} \cdot \frac{\partial \text{output}}{\partial \text{input}}$
  - each node needs upstream gradient + local derivative
- Reverse Topo Order
  - gradient of node depends on grad. from nodes after it
  - forward: inputs → intermediate nodes → output
  - backward: output → intermediate nodes → inputs
  - process nodes AFTER children
- left-to-right reduction order (from each)
  - If node influence loss from many paths, sum grads
    $\bar{v}_i = \sum_{j \in \text{children}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}$
- After backprop, only care abt adjoints for model parameters
  - ignore wrt labels/inputs → ~~$\frac{\partial L}{\partial x}$~~ ~~$\frac{\partial L}{\partial y}$~~
  - $W \leftarrow W - \eta \frac{\partial L}{\partial W}$ * $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b}$
- forward mode AD: one pass per parameter = slow
- reverse mode AD: one backward pass total
- gradient() for e-wise... $g = \frac{\partial L}{\partial y}$
  - Add: $y = a+b$ → $\frac{\partial L}{\partial a} = 1 \cdot g$, $\frac{\partial L}{\partial b} = 1 \cdot g$
  - Mult: $y = a \circ b$ → $\frac{\partial L}{\partial a} = g \circ b$, $\frac{\partial L}{\partial b} = g \circ a$
  - Div: $y = \frac{a}{b}$ → $\frac{\partial L}{\partial a} = g \circ \frac{1}{b}$, $\frac{\partial L}{\partial b} = g(-\frac{a}{b^2})$
  - matrix mul: $Y = AB$ → $\frac{\partial L}{\partial A} = (\frac{\partial L}{\partial y}^g) B^T$, $\frac{\partial L}{\partial B} = A^T (\frac{\partial L}{\partial y}^g)$
- $\sigma(x) = \frac{1}{1+e^{-x}}$ → $\sigma'(x) = \sigma(x)(1-\sigma(x))$
- softmax CE loss → $\frac{\partial L}{\partial x} = \text{softmax}(x) - e_y$ (OH label)
- $\ln x \to \frac{1}{x}$, $\sin x \to \cos x$, $\cos x \to -\sin x$, $x^2 \to 2x$
- Augment computation graph
  → immediate results, reuse structure
  → efficient, automatic gradient comp

DiSO2

- class Matmul(TensorOp)
  - def compute(self, a:array, b:array)
    - return a@b ← repeat → sum it
  - def gradient(self, out_grad:Tensor, node:Tensor)
    - lhs, rhs = node.inputs
    - return out_grad @ transpose(rhs, axes=None), transpose(lhs, axes=None) @ out_grad
- # undo reduction
  - grad = broadcast_to(reshape(out_grad, new_shape), input_shape)
- # undo broadcast
  - grad = summation(out_grad, axes=broadcasted axes)

→ A: (5,10,20), B: (20,30) } A@B = Y: (5,10,30)

↳ B reused for every matrix mult, many outgoing edges → need to sum partial adjoints

⇒ $\frac{\partial L}{\partial B} = \sum_{i=1}^{\text{batch}} (A_i^T \frac{\partial L}{\partial Y_i})$

⇒ (5,20,30) → (20,30)

⑩ Convolutional NN

- Conv2D has translational invariance
  - small shift in input → can still detect output
  - feature maps shift w/ it
  - convs reuse filters / weight sharing ⇒ better model, fewer param
- input spatial feature map
  - $p = \frac{k-1}{2}$, • output, input same size • odd kernel k (filter size) & and stride = 1 • padding = p
- output spatial feature map
  $\text{out\_dim} = \left\lfloor \frac{\text{in\_dim} + 2p - k}{s} \right\rfloor + 1$ ← separately for H, W
- reduce spatial resolution in convnets by factor of 2
  - stride = 2 in conv
  - max/avg pool (e.g. 2×2, stride = 2)
- BN for spatial feature maps
  - conv maps shape = [batch, channel, H, W]
  - normalize per channel over batch + spat location
  - ex: 1 channel takes stats over batch, H, W
- non-spatial MLP BN
  - normalize per feature over batch
- MNIST digit classifier vs convnet
  - MNIST flattened: [1,28,28] → 784 → fed → many param
  - MLP on flat, destroy 2D structure,
- convnet keeps spatial tensor
  - local receptive field, shares weights, less param
  - Pros: images, spatial, less param, capture shifts

04

- Model size & total num of model params
  - linear layer: [batch, in_feat] → [batch, out_feat]
  - batchsize = n, k = size of all intermediate act
    ↳ total number of elems in all intermediate acts
    n×k → # of layer outputs × batchsize × feat. per layer
  - linear: in_feats × out_feats + out_feats
  - layernorm: 2 × feat_dim
  - activation scales w/ batchsize

Layer	Shape (B,C,H,W)	Param
input RGB image	[B,3,224,224]	0
↓ (to get 8 channels)		
Conv2D (C_in=3, C_out=8, k=3, stride=2, padding=1)	[B,8,112,112]	8×3×3×3 + 8 ← C_out × C_in × k × k + C_out bias
↓		
BN2D (num_feat=8)	[B,8,112,112]	2×8 ← 8γ + 8β (scale, shift)
↓		
ReLU()	[B,8,112,112]	0
↓		
Conv2D(C_in=8, C_out=8, k=3, S=2, P=1)	[B,8,56,56]	8×8×3×3 + 8
↓		
BN2D (num_feat=8)	[B,8,56,56]	2×8
↓		
ReLU()	[B,8,56,56]	0
↓		
AvgPool2D (k=56)	[B,8,1,1]	0 ← to single feat vector for classification
↓		
Flatten()	[B,8]	0
↓		
Linear (in_feat=8, out_feat=1000, bias=True)	[B,1000]	8×1000 + 1000 ← C_in × C_out + C_out bias
↓		
CE loss	[1]	

$x = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

- $\hat{x} = \frac{x-\mu}{\sigma}$ ← norm
- $\sigma = \sqrt{\text{var}} = \sqrt{\frac{\sum (x-\mu)^2}{n}}$

LN: across features/rows
row 1: μ=1.5, σ=0.5 → norm → [-1,1]
row 2: μ=3.5, σ=0.5 → [-1,1]
LN(x) = [[-1,1],[-1,1]]

BN: across batch/cols
col 1: μ=2, σ=1 → norm → [-1,1]
col 2: μ=3, σ=1 → [-1,1]
BN(x) = [[-1,-1],[1,1]]

(batch, feat)

⇒ Conv2D: [B, channel, H, W]
- avg over B, H, W = BN2D
- avg over C, H, W = LN2D

$\text{LayerNorm}(x) = \frac{x-\mu}{\sqrt{\text{var}}}$
$\sigma^2 = \text{var}$, $\sigma = 2D$

Conv 2D
- k = $\frac{1}{4}$ (# of elem in kernel) $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ $x = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ ← 1 channel
- multi channel: C_out = num filt
  - shape of 1 conv2D filter: (C_in, k, k)
    weight bank: (num filt, C_in, k, k)
  - Z = conv2D(x, k=3, P=1, S=1, num filt=8)
    Z ∈ [C_out, H', W']
    ↳ input X size

up/down direction, L/R direction
Z[0,0,0]
channel 0, H=0, W=0

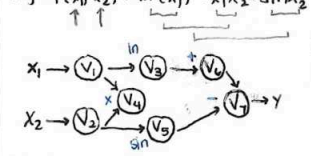
channel 1 | channel 0 → mult & add (channel 1, channel 0) ea.

max pool
$\frac{\partial l}{\partial x} = \begin{cases} \frac{\partial l}{\partial y} & \text{where max} \\ 0 & \text{else} \end{cases}$

avg
$\frac{\partial l}{\partial x} = \frac{1}{k^2} \cdot \frac{\partial l}{\partial y}$

Forward

- $y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin x_2$



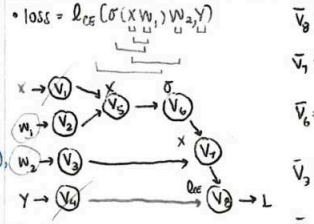
- list V's in terms of V & plug in
  - $V_1 = X_1$
  - $V_2 = X_2$
  - $V_3 = \ln V_1$
  - $V_4 = V_1 \times V_2$
  - $V_5 = \sin V_2$
  - $V_6 = V_3 + V_4$
  - $V_7 = V_6 - V_5$
  - $Y = V_7$

Backward (parent)

- start w/ output & look @ parent
  - $\bar{V}_1 = \frac{\partial y}{\partial v_1} = 1$ ← always
  - $\bar{V}_6 = \bar{V}_7 \frac{\partial V_7}{\partial V_6} = 1 \cdot \frac{\partial}{\partial V_6} V_6 - V_5$
  - $\bar{V}_5 = \bar{V}_7 \frac{\partial V_7}{\partial V_5} = \frac{\partial}{\partial V_5}(V_6 - V_5)$
  - $\bar{V}_4 = \bar{V}_4 \frac{\partial V_4}{\partial V_2} + \bar{V}_5 \frac{\partial V_5}{\partial V_2}$ ($\frac{\partial}{\partial V_2} V_1 \times V_2$, $\frac{\partial}{\partial V_2} \sin(V_2)$)
  - $\bar{V}_1 = \bar{V}_3 \frac{\partial V_3}{\partial V_1} + \bar{V}_4 \frac{\partial V_4}{\partial V_1}$

⇒ $\frac{\partial y}{\partial x_1} = \bar{V}_1$ $\frac{\partial y}{\partial x_2} = \bar{V}_2$

- loss = $\ell_{CE}(\sigma(X W_1) W_2, Y)$



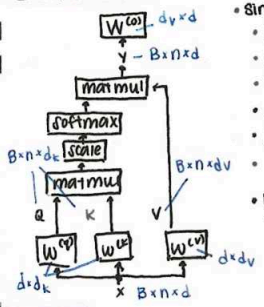
- $V_1 = X$
- $V_2 = W_1$
- $V_3 = W_2$
- $V_4 = Y$
- $V_5 = \text{matmul}(V_1, V_2)$
- $V_6 = \text{sigmoid}(V_5)$
- $V_7 = \text{matmul}(V_6, V_3)$
- $V_8 = \ell_{CE}(V_7, V_4)$

To minimize loss: $W_1 \leftarrow W_1 - \eta_1 \frac{\partial L}{\partial W_1}$

$W_2 \leftarrow W_2 - \eta \frac{\partial L}{\partial W_2}$

- $\bar{V}_8 = \frac{\partial L}{\partial V_8} = 1$
- $\bar{V}_7 = \frac{\partial L}{\partial V_7} = \bar{V}_8 \frac{\partial V_8}{\partial V_7} = \text{softmax}(x) - e_y$
- $\bar{V}_6 = \frac{\partial L}{\partial V_6} = \bar{V}_7 V_3^T = (\text{softmax}(x) - e_y) V_3^T$
- $\bar{V}_3 = \frac{\partial L}{\partial V_3} = V_6^T \bar{V}_7 = V_6^T (\text{softmax}(x) - e_y)$
- $\bar{V}_5 = \frac{\partial L}{\partial V_5} = \bar{V}_6 \odot \sigma'(V_6)$ $\sigma(V_5)(1-\sigma(V_5)) = V_6(1-V_6)$
- ~~$\bar{V}_1 = \frac{\partial L}{\partial V_1} = \bar{V}_5 V_2^T$~~
- $\bar{V}_2 = \frac{\partial L}{\partial V_2} = V_1^T \bar{V}_5$
  $= V_1^T [\bar{V}_6 \odot V_6(1-V_6)]$
  $= V_1^T [(\text{softmax}(V_7) - e_y) V_3^T \odot (V_6(1-V_6))]$

① Transformer



- Single-Head Attn
  - $Q = XW^{(q)}$ $B\times n\times d_k$
  - $K = XW^{(k)}$ $B\times n\times d_k$
  - $V = XW^{(v)}$ $B\times n\times d_v$
  - $A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$ $B\times n\times n$ (keep gradient healthy)
  - $H = AV$ $B\times n\times d_v$
  - $Y = HW^{(o)}$ $B\times n\times d$
- MHA
  - X → large learned projections → split Q,K,V into heads

* If $d_k = d_v = d_{model}$, ↳ Q,K,V same embedding width ↳ $\forall \in B\times n\times d$ SHA ↳ $Q,K,V,Y \in \mathbb{R}^{B\times n\times d}$

Meanings
- X = input token embedding
- B = batch size
- n = seq. length
- d/d_{model} = model embedding dim → in MHA, so total dim doesn't explode ↳ after concatenating, $H\cdot d_h = H\cdot \frac{d}{h} = d$ (width) → $n\times d$ matrix
- Q = query, token looking for
- K = what token advertise
- V = what info passed forward
- $W^{(q)}, W^{(k)}, W^{(v)}$ = learned weight matrices
- d_k = key/query dim → $d_k = d_q$ b/c inner-producted
- d_v = value dim → d_v can be diff
- QK^T = raw attn score $B\times n\times n$
- A = attn matrix $B\times n\times n$ ← self attn
- H = attn head output $B\times n\times d_v$
- $W^{(o)}$ = output projection matrix $d_v\times d$
- Y = final attn block output

Attn Matrix A
- how much ea. token attends to another
- A[i,j] one batch item
- softmax sum rows to 1

Self-Attn
- one seq → $Q = XW^{(q)}$, $K = XW^{(k)}$, $V = XW^{(v)}$
- ex: encoder self-attn, decoder masked self-attn, decoder-only
- ea. token attends to others in same seq.

Cross-Attn
- Tgt seq → $Q = X_{tgt}W^{(q)}$ (French) ? what src to look at?
- Src seq → $K = X_{src}W^{(k)}$, $V = X_{src}W^{(v)}$ (Eng)
- X_{tgt} $B\times n_{tgt}\times d$ — A $B\times n_{tgt}\times n_{src}$
- X_{src} $B\times n_{src}\times d$ — $Z = AV$ $B\times n_{tgt}\times d_v$
- Q $B\times n_{tgt}\times d_q$
- K $B\times n_{src}\times d_q$
- V $B\times n_{src}\times d_v$

Right-Shift Decoder Input
- next token prediction: decoder predict next token w/o seeing it
  - Y_{tgt} = ["Jaime", "chanter", <EOS>] ← X_{src} = ["I", "like", "to", "sing"]
  - X_{tgt} = [<BOS>, "Jaime", "chanter"]

Decoder Input	Predict
<BOS>	"Jaime"
<BOS>, "Jaime"	"chanter"
<BOS>, "Jaime", "chanter"	<EOS>

- Causal Attn Masks prevent decoder looking at future tokens
  1. compute raw scores: $S = \frac{QK^T}{\sqrt{d_k}}$
  2. create upper-triangular mask: j>i ← torch.ones
  3. S[:,j] = -∞ if j>i ← masked_fill -inf
  4. softmax(-∞) = 0 so future tokens get 0 attn ← dim 1
- use causal masks in decoder self-attn (not cross-attn)
  - ↳ decoder self-attn looks at partial tgt seq. SVN
  - ↳ cross attn looks from tgt to src & full src alr known
- Eng src → Encode → encoded Eng representation (full Eng)
- French so-far → Decode → next French token

Bottleneck: quadratic in seq. len. ⇒ 2× length → 4× compute cost
- Attn_matrix: QK^T b/c self-attn creates n×n for seq. len. n
- MHA calculates $[n_{tgt}, n_{src}]$ attn_score matrix
- $QK^T \sim O(n^2d)$, $AV \sim O(n^2d)$ → attn_block $\sim O(n^2d)$

Architectures
- Encoder-only (representation of input) ← fill in blank
  - text classification, sentiment, BERT, img class. w/ ViT
- Encoder-Decoder (seq-to-seq) cat ↔ it
  - translation, summarization, Q/A, "attn is all you need"
- Decoder-only (autoregressive gen)
  - GPT, chatbot, next-token prediction

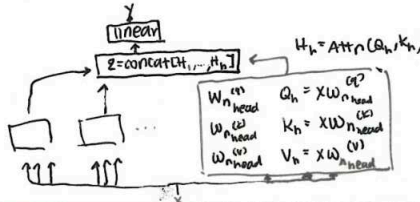
Tokens
- <BOS> Beg. of seq.; first decoder input } auto reg. translation
- <EOS> generation should stop
- <PAD> make seq. same len. in batch
- <UNK> tokenizer sees smth unknown
- <CLS> represent whole seq. for classification

Classification → Logits = $Y_{CLS}W$
- [<CLS>, $x_1, x_2, \ldots x_n$]
- after encoder, take output at <CLS> & feed into classifier
- <CLS> has global info. & BERT trains ontop of it

Agg/Pooling → Logits = $Y_{pooled}W$ (Y dim=1)
- instead of <CLS>, agg over output tokens (max, mean) into single embedding

Attn Pooling
- learns which token matters most for tasks and gives them more weight when pooling
- $Y \in B\times n\times d \Rightarrow z \in B\times d \Rightarrow zW \in B\times \text{num\_classes}$ (classifier)

Positional Encodings
- add order info to token embeddings before attn
- "I am happy" vs "am I happy"



② Visual Transformer
- CNN > ViT on small/med training dataset b/c strong inductive bias for spatially-local features
- ViT > CNN for large dataset
- Inductive bias = what model arch. builds in on useful patterns as
  - ↳ CNN = local spat. pattern, nearby pixels w/ filter, some translation invariance
  - ↳ ViT = img as seq. of tokens, learn local + globally
- ViT turns img into seq. of img patches
  - 224×224 RGB img w/ 16×16 patch
  - ↳ split into patch → raster order → patch flatten into vector → project to token embed. → add pos. embed → feed into encoder
  - ↳ $\frac{224}{16}\times\frac{224}{16} = 14\times14 = 196$ patch tokens
- pretrain ViT b/c less built-in assumptions not random weight
  - on lrg data set → learn general
  - then, finetune on task w/ strong representation

③ Masked Auto Encoder (self-supervise)
- mask patches → train to reconstruct → throw away decoder → use pretrained encoder for classif.
1. visible image patches → encoder for strong repres.
2. visible patch embeddings + mask token → decoder to predict reconstructed img patches
3. use transformer encoder for img classification
- pass unmasked patches to encoder:
  - ↳ cheaper & avoids train/test mismatch input drift
- Pretraining task = self-supervised reconstruction, fill in missing patches
- Finetuning task = supervised downstream, img classification
- toss decoder! keep encoder

④ Scaling Up Training (multiGPU, distributed training)
- Horizontal scaling = add more GPUs
- Vertical scaling = make GPU stronger (limited)
- Distributed Data Parallel (DDP)
  - copy full model on every GPU, split data, sync grad.
  - ↳ after bkwds, avg GPU grads + apply same
  - ↳ copy initial weights + grad synchronization
  - use: model fit 1 GPU, too slow training, high throput ~10× faster but communication overhead
- World_size = total # GPUs
- rank = ID of worker (GPU 1 = rank 1)
- torchrun = start distr. training
- DDP wrapper so Pytorch handle sync
- Fully Sharded Data Parallel (FSDP)
  - shard/split params, grads, optimizer state
  - NOT full model on GPUs
  - use: can't fit on 1 GPU, big models
- Adam stores ⇒ grad + 1st moment + 2nd moment = 3N extra values = optimizer state matters

DDP	FSDP
• Full model on GPUs	• Shard model
• Faster throughput	• lower memory use
• full duplicate per GPU	• model split across GPUs
• model fits on 1 GPU	• model doesn't fit onto 1
• waste mem. duplicating weights	• more communication overhead
• gradient all-reduce (avg grads)	• param/grad/opt. shard communication

* need cross device comm. to keep 1 shared model

- all-reduce combines values across devices
- NCCL = Nvidia's library for GPU comm.
  - ↳ ring all_reduce = GPUs pass gradient chunks around in a ring
- single-node multi-GPU
  - ↳ 1 machine w/ many GPUs = faster comm.
- multi-node multi-GPU
  - ↳ ranks assigned: GPU 1, rank 0-7...
  - ↳ slower communication but larger training
- DDP: effective total batch size = local batch size × # of GPUs → N× GPUs → N× effective batch size
- linear scaling rule
  - batch size × N → learning rate × N
- learning rate schedule
  - small (avoid expl. grad) → increase → decay (details)

⑤ Large-scale pretraining
- supervised = training ex has input + human tgt label
  - ex: img classification, obj. detection, translation
- unsupervised = data w/o labels
  - ex: k-means cluster, NN group, dim reduction
- self-supervised = labels created from input
  - NLP masked-word prediction, GPT next-token prediction, MAE img pretraining (img patches)
- 1) Pretraining
  - broad dataset to learn general representation
  - ViT + imgset, MAE + reconstruct patch, GPT internet
- 2) finetuning
  - use pretrained model + train on specific task
  - MAE img class, GPT chat safety, (head)
  - freeze pretrained, train classifier = linear probing
  - strong initialization → specialize for downstream
- GPT pretrains: lrg-scale, human-written text (later larger + noisier)
  - ↳ task = next-token prediction + causal attn mask
  - ↳ one forward/backward pass can compute loss for all

⑤ Recommendation System
- tiny corpus = 1 stage ok
- Stage 1: candidate generation/retrieval (precision)
  - billions → thousands w/ embeddings + NN search + recall
- Stage 2: ranking
  - rerank smaller set + precision priority
- nearest neighbor embedding search
  - query as embed. vector, compare (dot prod/cosine), return most similar
- Img embed / Text embed
  - ConvNet/Resnet = use after avg pool + before logits
  - ViT / Transformer encoder = [CLS] or agg output tokens
  - decoder only = final or pooled hidden state, causal trick
  - encoder only = cleaner, encode bidirectionally
- Approximate NN
  - NN linear search is slow & ANN is 'good enough'
  - locality sensitive hashing
  - HNSW = graph over embed & search instead of comparing against every item
  - distributed ANN = shards ANN index across N machines
    - ↳ scatter/gather architecture → agg results
- Two-Tower Model
  - Ex: user tower → user feature → MLP → user embed; item tower → item feature → ConvNet/ViT → embed
  - ⇒ similarity(user, item) = dot prod or cosine similarity
  - User engagement log: ⊕ ex = clicked on item; ⊖ ex = randomly sample not engage
  - binary loss classification = did user click?

CLIP
- text + image tower; img-txt pairs; dot prod. similarity
- ⊕ = match img-caption; ⊖ other pairs
- * align diff embeddings onto common embed. space
- during train, ⊕ pairs along diagonal of similarity matrix

Ranking model
- take retrieval candidate set to final order
- pointwise ranking: or p(user clicks), scores one user-item pair at a time
- use demographics → text topic features
- Domain shift = data distribution changes btwn training & deployment
  - ↳ ex: ImageNet & pinterest fashion
  - ↳ fix: fine-tune on target,
- Metric learning: train embeddings so similar = close & diff things = far
  - obey distance metrics + learn clusters
- Anchor (item/user), Positive (item clicked), neg. (item ignored)
  - ↳ triplet loss pushes: dist(anchor, pos) < dis (anchor, neg)
  - ↳ user engagement data = crucial

- pretrained LLM = base model & need fine-tuning
- SFT: input-output pairs to teach instruction/format
- RLHF: human preference
  - ⇒ purpose: helpful/truthful/unharmfulness
- finetuning data from: humans, auto running math/code, stronger models generate instruction-following data for weaker models
- GPT-3+ = FSDP
  - can't fit into 1 machine, param/grad/optimizer split
  - (DDP would duplicate model + run out of memory)

⑦ Multi-modal Models

- CLIP = contrastive language-img pretraining
  - learned shared embedding space where matching img-text are close
  - 2 towers: img encoder – img → img embedding
    text encoder – txt → txt embedding
  - alignment score = dot product btwn embeddings
    - main diagonal = positive pairs, correct match
    - off-diagonal: mismatch img-caption
  - contrastive learning: correct pair = high dot prod.
    wrong pair = low
  - need lrg enough batchsize so negatives diverse
  - lrg dataset of (img, txt caption) pairs
- CLIP vs GPT zero-shot classification
  - Both: do classification w/o training new classifier, lrg-scale pretraining, use natural language
  - Diff:

	CLIP	GPT
mechanism	compare embedding	generate text
output	high similarity	generated answer
for modalities	img-txt match, img classification img+txt	txt classification, QA, summarize, reason. txt
score ex:	Dot prod / cosine sim	token probs / generated response
	img closest to "photo of dog	"classify this..."

- for audio-text, replace img encoder w/ audio encoder
  - high quality captions improve alignment
- LLaVA - vision-language model
  - takes text-only LLM & make it see imgs
  - Pretrained vision encoder (eg. ViT/CLIP) to convert img into visual tokens → into input seq w/ txt
  - pass ALL encoder output token embeddings
    - img token have more detailed semantics
  - Projection layer: vision encoder output dim → LLM embedding dim
  - img-txt instruction-following data

DIS11

Audio to ML

- raw waveform (long): WaveNet
  - Pros: orig signal, model fine details, good for TTS
  - Cons: long, expensive, hard for transformers
- frequency: Mel spectrogram / MFCC
  - Pros: shorter, structured, meaningful, image-style, good for classification, speech, etc.
  - cons: lose raw waveform detail, vocoder for audio
- Audio tokens / codebook IDs (VALL-E)
  - pros: transformer style, seq of tokens, speech generation + voice cloning
  - cons: separate tokenizer, long / compute heavy

DIS08

- Block matrix identity
  - $K = XW^{(K)} = [XW_0^{(K)} | XW_1^{(K)}] = [K_0 | K_1]$

$K \in \mathbb{R}^{B \times n \times d}$ → [B, n, num_heads, d_h]

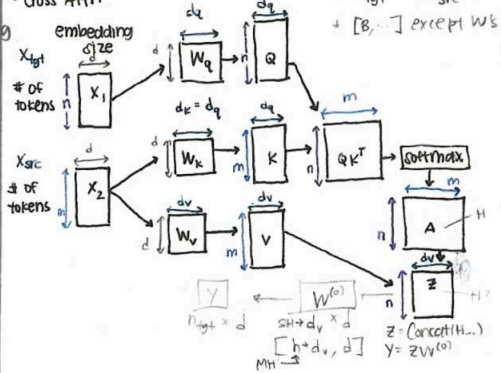
d = num_heads × d_h

- compute full attn matrix & split (per head)
- If $n_{heads}=2$, seq len n=2, hidden dim d=4, $d_h=2$

$Q = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$ → $Q_0 = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}$, $Q_1 = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}$ $Q = \text{concat}[Q_0, Q_1]$

DIS09

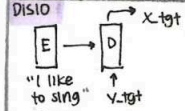
- Cross Attn



* n_{tgt} or n_{src}
+ [B, ...] except Ws

- Self Attn: A ∈ [B, n, n]
- Cross Attn: A ∈ [B, n_{tgt}, n_{src}]
- 1) Q length → output length
  2) K/V length → 2nd dim of attn
  3) If Q & K same seq.
  4) +B to all tensors, not weights
  5) add h for multi-head

DIS10



- cross token interactions only at MHA
- other layers are point-wise
- OR flatten X_src into [n_src * d] so MLP sees (bad, hardcodes seq. length)
- ViT
  - pass high-res into ViT w/ longer seq. length.
  - inference runs, but w/ domain shift

- 2 tower

feature embed. → DNN → user embed ↘ (cosine) similarity → P(user likes) ↓ BCE

feature embed → DNN → item embed ↗ (one)

DIS12

- 1T = 10^{12} bytes
- float16 = 2bytes
- 1 GB = 10^9 bytes
- To train GPT-4 & Adam: 1.8T model param)

grad, running avg, # of model param (magnitude) float

$\frac{3 \cdot N \cdot 2\text{bytes}}{10^9} = \frac{3 \cdot 1.8 \cdot 10^{12} \cdot 2}{10^9} = 10800$ GB

3600 + 10800 = 14400 GB

model weights

MT SP26

3a. input activation to sigmoid have lrg magnitude → exploding gradient, model instable
b. loss differentiable
c. MNIST + CrossEntropy
7b. learning rate only helps exploding gradient
d. big bias term to linear layer → model complexity collapse into single linear layer

Normalization

- LN1D: $X = \begin{bmatrix} 8 & 2 \\ 0 & 4 \end{bmatrix}$ ⇉

row 1: $\mu = \frac{8+2}{2} = 5$, $\sigma = \sqrt{\frac{(8-5)^2+(2-5)^2}{2}} = \sqrt{9} = 3$ → $\left[\frac{8-5}{3}, \frac{2-5}{3}\right] \to [1, -1]$

row 2: $\mu = \frac{0+4}{2} = 2$, $\sigma = \sqrt{\frac{(0-2)^2+(4-2)^2}{2}} = \sqrt{4} = 2$ → $\left[\frac{0-2}{2}, \frac{4-2}{2}\right] \to [-1, 1]$

Computer Vision

Layer	H_{out}	shape (B,C,H,W)	Param
x	$\left[\frac{H+2p-K}{S}\right]+1 = \left[\frac{224+2(1)-7}{1}\right]+1 = 220$	[B, 1, 224, 224]	0
Conv2d1 (C_in=1, C_out=2, k=7, s=1, pad=1, bias=True)		[B, 2, 220, 220]	2×1×7×7 + 2
BN2D1 (num_feat = C)		[B, 2, 220, 220]	2×2
ReLU1	$H_{out} = \left[\frac{220+2(1)-3}{4}\right] = 55$	[B, 2, 220, 220]	0
Conv2d2 (C_in=2, C_out=4, k=3, stride=4, pad=1, bias=True)		[B, 4, 55, 55]	4×2×3×3 + 4
BN2D2 (num_ft = C)		[B, 4, 55, 55]	2×4
ReLU2		[B, 4, 55, 55]	0

conv2d params = $C_{out} \times C_{in} \times K_H \times K_W + C_{out}$ (if bias)

batchNorm2D = 2 × num_feats (Cout from conv2D)

For $z = MSE(x, y) = \frac{1}{d}\sum_{i=1}^{d}(x[i]-y[i])^2$

$\frac{\partial z}{\partial x} = \frac{2}{d}(x-y)$

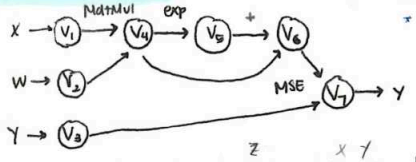
$\frac{\partial z}{\partial y} = -\frac{2}{d}(x-y)$

$z = A @ B$

$\frac{\partial loss}{\partial z} @ \frac{\partial z}{\partial A} = \frac{\partial loss}{\partial z} @ B^T$

$\frac{\partial loss}{\partial z} @ \frac{\partial z}{\partial B} = A^T @ \frac{\partial loss}{\partial z}$

Computation Graph



* look forward one arrow
* write back one deriv

1. start from symbolic adjoint pattern
2. substitute local deriv.

$V_1 = X$, $V_2 = W$, $V_3 = Y$

$V_4 = V_1 @ V_2$, $V_5 = \exp(V_4)$, $V_6 = V_4 + V_5$

$V_7 = MSE(V_6, V_3) = \frac{1}{d}\sum_{i}^{d}(V_6[i]-V_3[i])^2$ (Z, X, Y)

$\frac{\partial z}{\partial x} = \frac{2}{d}(x-y)$

$\bar{V}_7 = 1$

$\bar{V}_6 = \bar{V}_7 @ \frac{\partial V_7}{\partial V_6} = \frac{2}{d}(V_6 - V_3)$

$\bar{V}_5 = \bar{V}_6 @ \frac{\partial V_6}{\partial V_5} = \frac{2}{d}(V_6 - V_3) @ 1$

$\bar{V}_4 = \bar{V}_6 @ \frac{\partial V_5}{\partial V_4} + \bar{V}_6 @ \frac{\partial V_6}{\partial V_4}$

$\bar{V}_3 = \bar{V}_7 @ \frac{\partial V_7}{\partial V_3}$

$\bar{V}_2 = \bar{V}_4 @ \frac{\partial V_4}{\partial V_2}$

$\bar{V}_1 = \bar{V}_4 @ \frac{\partial V_4}{\partial V_1}$

$\frac{\partial V_6}{\partial V_5} = \frac{2}{d}(V_5 + V_5)$

$= \frac{\partial}{\partial V_5} V_5 = 1$

$\frac{\partial V_5}{\partial V_4} = \frac{\partial}{\partial V_4}(\exp V_4)$ $e^{V_4} = e^{V_4}$

$\frac{2}{d}(V_6 - V_3) @ e^{V_4} + \frac{2}{d}(V_6 - V_3) \cdot 1$